

Large Language Model-based Web Navigating Agent

Bochen Wang

Carnegie Mellon University
Pittsburgh, PA 15213

`bochenw@andrew.cmu.edu`

Aijia Lei

Carnegie Mellon University
Pittsburgh, PA 15213

`aijial@andrew.cmu.edu`

Wang Xiang

Carnegie Mellon University
Pittsburgh, PA 15213

`wxiang@andrew.cmu.edu`

Abstract

The integration of Large Language Models (LLMs) with web navigation capabilities represents a significant advancement in natural language processing. This field addresses the challenge of enabling AI systems to autonomously browse on the web, performing complex tasks like searching, summarizing, and data retrieval, similar to human users. This paper focuses on autonomous web navigation agents, with particular attention to improving the recent developments in LLM-based approaches. By leveraging enhancements such as contextual expansion and chain-of-thought reasoning, our approach improves the accuracy and adaptability of these agents. Comprehensive evaluations on established benchmarks demonstrate significant improvements and achieve a high success rate. The results provide a foundation for the advancement of autonomous web browsing capabilities with practical and scalable solutions.

future work.

1 Introduction

In recent years, Large Language Models (LLMs) have demonstrated impressive capabilities in natural language processing, extending their impact into tasks like web navigation, where users interact with online platforms through natural language instructions. The rapid evolution of the internet has transformed it into an indispensable resource for information retrieval, e-commerce, and daily task management. This transformation has spurred significant interest in developing autonomous web browsing agents capable of navigating complex web environments and executing tasks based on natural language instructions. Such agents hold the potential to revolutionize human-computer interaction by automating routine tasks and providing seamless access to web-based services.

The advent of LLMs marked a significant advancement in autonomous web navigation. Mod-

els such as GPT-4(OpenAI et al., 2024) and Claude-2(Wu et al., 2023) have demonstrated remarkable proficiency in understanding and generating human-like text, facilitating more sophisticated interactions with web interfaces. Nevertheless, the substantial computational resources required for deploying these models have prompted the development of smaller, more efficient alternatives, including LLaMA-2-7B(Touvron et al., 2023) and ChatGLM3-6B(GLM et al., 2024), which offer comparable performance in specific domains.

In this paper, we present an improved approach to autonomous web navigation that leverages the strengths of LLMs while addressing the limitations of previous methods. Our methodology integrates several interconnected stages, beginning with data preparation using the dataset to provide a comprehensive and realistic foundation for training and evaluation. We refined few-shot prompting and added context by transforming complex HTML structures into actionable instructions, incorporating enhancements such as contextual expansion and chain-of-thought reasoning to improve the model’s comprehension and task execution. The LLM configuration is optimized through hyperparameter tuning, ensuring robust and precise action generation. Actions are parsed and mapped for seamless interaction with web environments, while iterative feedback mechanisms further refine performance. By maintaining high efficiency and reducing computational requirements, our approach enhances the agent’s ability to generalize across diverse web environments without relying on task-specific hierarchical structures, thereby improving scalability and adaptability. Through comprehensive evaluations on established benchmarks, we demonstrate the effectiveness of our approach in advancing the capabilities of autonomous web browsing agents.

2 Related Work

Developing a comprehensive web browsing agent involves integrating various components, including language models for decision-making and HTML parsers for environment observation. Equally important is the establishment of robust evaluation benchmarks to assess the agent’s performance. The earliest approaches, emerging around 2015 to 2018, focused primarily on supervised learning techniques. For instance, Web2Text(Vogels et al., 2018) navigates web elements by constructing neural network classifier using Convolutional Neural Networks (CNNs)(O’Shea and Nash, 2015) to identify main content on web pages based on structural and textual features of consecutive text blocks. But they face significant challenges when dealing with dynamic layouts.

Large Language Models (LLMs): Large Language Models (LLMs) have demonstrated extensive capabilities across various natural language processing tasks. Notable examples include GPT-4 (Achiam et al., 2023), LLaMA-2 (Touvron et al., 2023), ChatGLM (GLM et al., 2024). Despite their impressive performance, the substantial computational resources required for deploying these models have prompted the development of smaller, more efficient alternatives. Open-source projects like LLaMA-2-7B(Touvron et al., 2023) and ChatGLM3-6B(GLM et al., 2024) have shown that smaller models can achieve performance comparable to larger LLMs in specific domains.

Benchmarks for Web Navigation: Evaluating web browsing agents necessitates diverse benchmarks that reflect real-world tasks. Mini-WoB++(Shi et al., 2017) offers a collection of simulated web environments designed to test models’ abilities to interact with webpage components. However, as the complexity of web operations has increased, more comprehensive benchmarks have been introduced. Mind2Web(Deng et al., 2024) provides an offline evaluation set for complex web browsing tasks, offering straightforward metrics for model assessment. In contrast, WebArena(Zhou et al., 2024) is based on real websites and creates multiple virtual environments, employing various evaluation methods to assess task completion rates, making it more suitable for real-world task evaluation.

Agents for Web Automation: Integrating LLMs with web environments has led to the devel-

opment of agents capable of performing a range of tasks. Early work, such as WebGPT (Nakano et al., 2021) and WebGLM (Liu et al., 2023), focused primarily on question-answering (QA) tasks, utilizing internet resources to respond to user queries. Recent advancements have shifted towards executing complex operations and interactive tasks. A fundamental aspect of web browsing tasks involves a comprehensive understanding of the Document Object Model (DOM). For instance, DOM-Q-NET(Jia et al., 2019) employs a graph neural network to represent the tree-structured HTML of web pages, facilitating reinforcement learning-based web navigation. Similarly, DocStruct(Wang et al., 2020) utilizes multimodal methods to extract hierarchical structures in forms, leveraging semantic content, layout information, and visual images. However, these approaches often depend on task-specific hierarchical structures of the DOM, which may limit their scalability and adaptability to diverse web environments. In contrast, WebAgent(Gur et al., 2024) utilizes HTML-T5 to process the observation space’s content, including HTML, previous operations, and user instructions. It employs the Flan-U-PaLM model to generate code for controlling webpages, demonstrating excellent web browsing performance. However, deployment challenges arise due to the model’s substantial size, at a 540B scale.

3 Methodology

The methodology centers on designing, implementing, and refining a Large Language Model (LLM)-based web navigation agent. The overall process is meticulously structured into distinct stages: dataset preparation, prompt engineering, LLM utilization, action generation, and webpage interaction. Each stage is intricately interlinked, ensuring a coherent workflow to achieve optimal navigation performance. Figure 1 illustrates the overall structure of the proposed methodology. Below is a detailed description of each stage and their interrelations.

3.1 Overview

The methodology’s stages are inherently interconnected, forming a cohesive pipeline for developing an LLM-based web navigation agent. Starting with dataset preparation, the pipeline builds a foundation for task-specific training and eval-

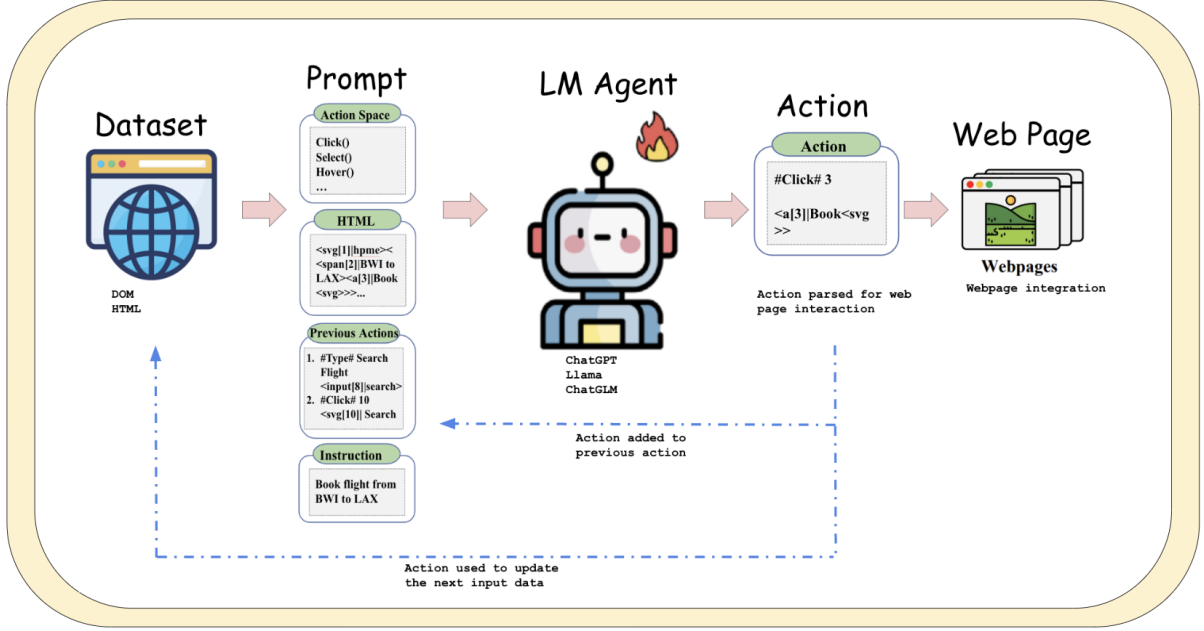


Figure 1: Framework

uation. The next stage, prompt engineering, bridges raw data with actionable instructions by transforming complex HTML structures into understandable directives for the LLM. Subsequent stages, including model configuration and action generation, ensure precise interpretation of instructions and accurate execution of web interactions. Finally, the reward framework and webpage interaction stages provide iterative feedback, closing the loop for refinement and performance enhancement. This interconnected pipeline reflects a holistic approach, where each stage enriches the subsequent ones, enabling robust and intelligent web navigation.

3.2 Data Preparation

We leveraged the Miniwob++ dataset, a benchmark dataset tailored for web-based interaction tasks. This dataset comprises a collection of HTML structures and their corresponding Document Object Model (DOM) lists. The dataset serves as a foundation for training and evaluating the LLM-based navigator, providing a realistic and diverse range of web navigation tasks.

The Miniwob++ dataset was selected due to its comprehensive representation of dynamic web elements and intricate user interactions. Each task in the dataset is encapsulated in a structured format, detailing the webpage’s HTML, the specific user instruction, and the action required to complete the task. This data representation facilitates

both model training and prompt design, ensuring consistency and alignment across stages.

3.3 Prompt Engineering

The prompt design is a critical step that bridges the dataset and the LLM. The structure of the prompt is shown in figure 2. The initial design of the prompt included the following components:

- **HTML**: The raw HTML code of the webpage.
- **Action Space**: A set of permissible actions the agent could execute.
- **Previous Action**: The last action performed, designing for multi-action tasks and providing continuity.
- **Instruction**: The explicit task description or user goal.

Given the complexities of web navigation, the initial prompt design underwent three major revisions to enhance model understanding and task execution:

Few-shot Learning Integration To provide the model with explicit examples of successful task completions, we augmented the prompt with a set of curated few-shot examples. These examples demonstrated the correct mapping of instructions to actions in various contexts, reinforcing the LLM’s ability to generalize across tasks.

Chain-of-Thought Reasoning Recognizing the importance of step-by-step reasoning in com-

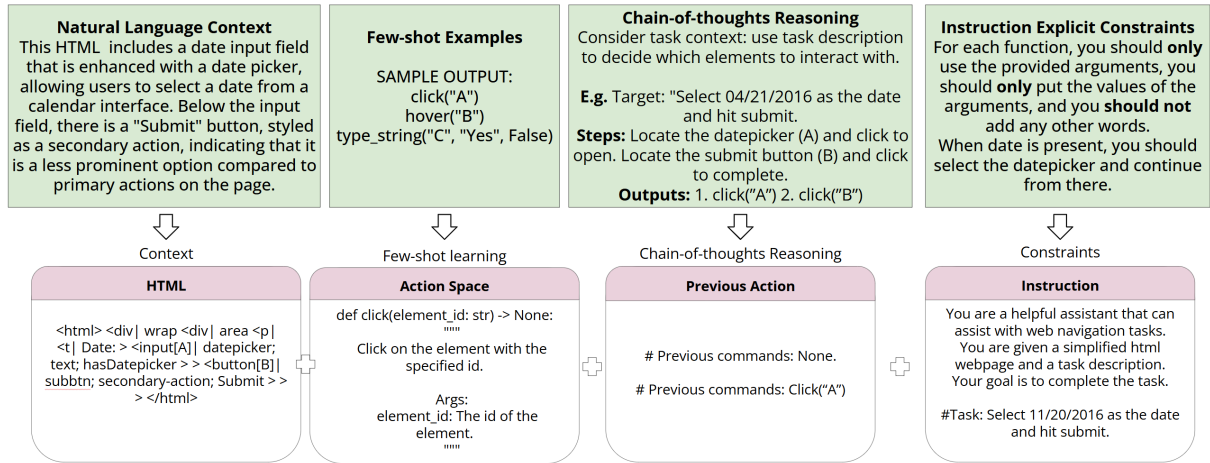


Figure 2: Framework

plex navigation tasks, we incorporated a chain-of-thought approach. This entailed including examples that explicitly outlined the logical steps taken to solve a task, guiding the model to adopt a systematic approach to problem-solving.

HTML Context Expansion To address the model’s challenges in comprehending HTML structure, we proposed an integration of context expansion. The HTML Context Expansion leverages a lightweight auxiliary model to translate raw HTML structures into natural language descriptions, providing the LLM with an enriched semantic understanding of the webpage. This innovative addition significantly improves the agent’s ability to interpret and navigate complex web environments.

The lightweight model functions as a mediator, processing the hierarchical and technical nature of HTML into descriptive narratives that the LLM can easily comprehend. For instance, it translates a `<div>` structure containing a button into, “A blue button labeled ‘Submit’ is located near the bottom right of the page.” Such translations provide context about element types, positions, and functionalities that are otherwise opaque in raw HTML.

By distilling raw HTML into natural language, the model bridges the gap between technical markup language and human-readable instructions, aiding the LLM in contextualizing webpage elements. The natural language output includes relationships between elements, such as parent-child hierarchies and sibling nodes, which are essential for tasks requiring multi-element interaction. The auxiliary model abstracts intricate HTML at-

tributes into simplified yet informative descriptions, enabling the LLM to focus on task reasoning rather than low-level parsing.

3.4 Large Language Model Configuration

We utilized a state-of-the-art LLM, including GPT-4o, LLaMA2, and ChatGLM, to interpret the prompts and generate appropriate navigation actions. Extensive experimentation was conducted to optimize the temperature hyperparameter, which controls the randomness of the model’s outputs. Through iterative testing, we identified an optimal temperature value that balanced exploration and precision in action generation.

Additionally, the model’s performance was evaluated across various configurations, ensuring robustness and adaptability to diverse tasks within the Miniwob++ dataset. These evaluations were essential in determining the LLM’s suitability for real-world web navigation challenges.

3.5 Action Generation

Once the LLM generated a response based on the prompt, the output was parsed and mapped to actionable commands.

Parsing The model’s textual response was analyzed to extract the intended action. This included identifying target elements (e.g., buttons, input fields) and the specific interaction type (e.g., click, type, scroll).

Mapping The parsed action was translated into a format executable on the webpage, ensuring compatibility with the browser’s navigation framework.

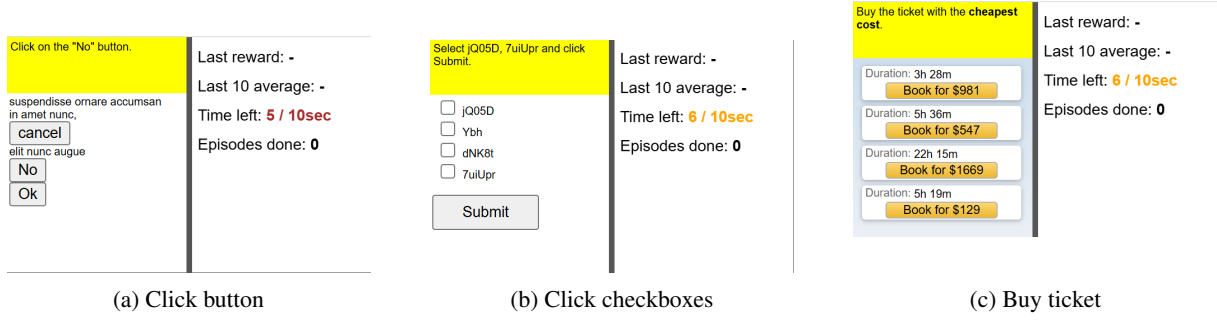


Figure 3: Examples of tasks from MiniWob++ Dataset

Reward Framework Enhancement: To improve the evaluation of the web navigation agent’s performance, we developed a hybrid reward framework that combines regex-based validation with a step-wise scaling mechanism. This approach not only ensures precise pattern matching for task-specific validation but also dynamically adjusts rewards based on task complexity and progress.

Regex patterns form the core of the reward framework, allowing precise evaluation of task outcomes at each step. These patterns capture key HTML elements, attributes, and states, ensuring alignment with expected results. For instance: `<input.*?id="email".*?value=".*?">` confirms that the agent entered a value into the "email" field, while the expression `<div.*?class="success">.*?</div>` verifies that the task concludes with the correct success message on the webpage.

Step-wise scaling mechanism adjusts rewards dynamically based on task progression and difficulty. Each task is divided into predefined stages, with rewards increasing as the agent progresses to more challenging or significant steps. For example, identifying a clickable button might earn 5 points, but triggering a correct submission earns 50 points, reflecting the task’s completion significance.

The framework combines regex validation with step-wise scaling to deliver rewards that are both granular and adaptive. At each step, regex patterns confirm success or failure. Then, scaling adjusts the reward magnitude based on the stage’s importance. For example, entering a correct value into a field validated by regex earns 10 points, while successfully completing the task after multiple intermediate checks results in a cumulative reward, weighted by the scaling mechanism.

This combination ensures the agent receives ac-

curate feedback for individual actions while encouraging optimal task completion by rewarding progress in a structured, meaningful way.

3.6 Webpage Interaction

The final stage involved executing the generated actions on the webpage and capturing the resultant state as a screenshot. This step ensured visual verification of task completion and provided feedback for further model refinement. The screenshots served as a record of the agent’s performance, enabling detailed error analysis and debugging.

Figure 3 illustrates examples of task operations, represented as screenshots from navigating various tasks in Miniwob++ datasets.

4 Experiments and Results

4.1 Dataset

MiniWoB++ by [Humphreys et al. \(2022\)](#) is a comprehensive suite of over 100 web-based environments, 56 tasks, designed to evaluate and train models on diverse web interaction tasks, ranging from simple actions like clicking buttons to complex activities such as form filling and information extraction. By providing programmed rewards for each task, it facilitates the application of standard reinforcement learning techniques, making it a benchmark for assessing the capabilities of large language models in navigating and interacting with web interfaces.

4.2 Experimental Set Up

The evaluation of models are performed on a T4 GPU within a Google Cloud Virtual Machine (VM).

4.3 Models

Our baseline model reproduction focuses on three large language models: GPT-4o ([OpenAI](#),

2024), LLaMA2-7B (Touvron et al., 2023), and ChatGLM3-6B (GLM et al., 2024). These models were chosen for their strong performance in NLP tasks and their alignment with our research objectives and computational considerations.

GPT-4o (OpenAI, 2024), an enhanced version of GPT-4, serves as the primary baseline due to its state-of-the-art performance in web navigation tasks, as demonstrated in the AutoWebGLM study (Lai et al., 2024). It offers a benchmark for maximum achievable performance, providing critical insights into the upper limits of current models.

LLaMA2-7B (Touvron et al., 2023), with its smaller parameter count of 7 billion, represents a more efficient alternative, highlighting the trade-offs between performance and computational cost. This model addresses the growing demand for resource-efficient solutions in practical applications.

Lastly, ChatGLM3-6B (GLM et al., 2024), the latest in the ChatGLM series, incorporates diverse training data and improved strategies. It was selected to evaluate open-source alternatives to AutoWebGLM’s baseline and explore the capabilities of models with smaller parameter counts in web navigation tasks.

4.4 Metric

Performance was assessed using the Average Score, calculated as the mean reward across all test cases. The reward, predefined by MiniWoB++ (Humphreys et al., 2022), starts at 0 for each episode and is adjusted within a range of (-1, 1) based on the agent’s actions in a given state, reflecting the outcomes of those actions in terms of accuracy.

The accuracy score is calculated using the formula:

$$\text{Accuracy Score} = \left(\frac{C}{B} \right) \times \left(\frac{1}{T} \right) \times 100\% \quad (1)$$

where:

C = Number of Correct Actions

B = Number of Batches

T = Total Tasks

4.5 Results

4.5.1 Temperature Tuning Across Models

Figure 4 highlights our accuracy of three models—GPT-4o, LLaMA2-7B, and ChatGLM-

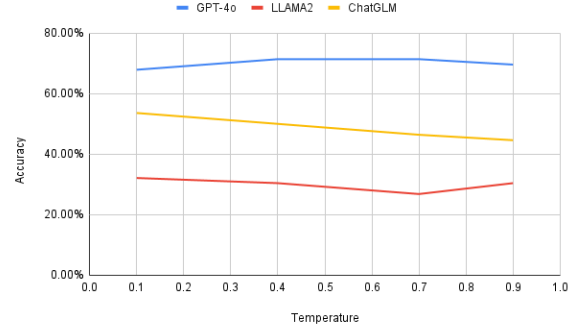


Figure 4: Temperature Tuning Across Models

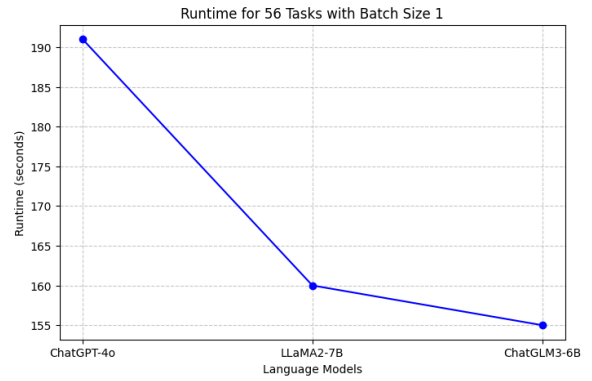


Figure 5: Runtime Across Models

6B—across different temperature values ranging from 0.0 to 1.0. GPT-4o consistently outperforms the other models, maintaining an accuracy rate near 70% across all temperatures. In contrast, LLaMA2-7B exhibits a declining trend, starting at approximately 30% accuracy and falling slightly as the temperature increases. ChatGLM-6B performs worse overall, with accuracy consistently below 20%, indicating limited robustness across varying temperature settings.

4.5.2 Runtime Across Models

Figure 5 compares the runtime of the three models for completing 56 tasks with a batch size of 1. GPT-4o is the slowest, taking approximately 190 seconds, followed by LLaMA2-7B at around 160 seconds, and ChatGLM-6B being the fastest with a runtime of approximately 155 seconds. This demonstrates that while GPT-4o delivers higher accuracy, it comes at the cost of increased computational time.

4.5.3 Success Rates for Modifications

Table 1 summarizes our success rates for different modifications applied to the models: Baseline, Context, Reward, and Context+Reward.

Model	Baseline	Context	Reward	Context+Reward
ChatGPT-4o	44.1%	50.3%	46.7%	71.4%
Llama2-7B	23.21%	28.9%	25.8%	32.1%
ChatGLM-6B	14.3%	18.5%	16.2%	53.6%

Table 1: Success Rates for Different Modifications across Models

Baseline Performance GPT-4o achieves the highest baseline success rate of 44.1%, followed by LLaMA2-7B (23.21%) and ChatGLM-6B (14.3%).

Context Integration Context modifications improve success rates for all models, with GPT-4o reaching 50.3%, a notable increase compared to LLaMA2-7B (28.9%) and ChatGLM-6B (18.5%).

Reward Reward that modify the calculation of reward integration further enhances performance, with GPT-4o achieving 46.7%.

Combined Context and Reward The combination of context and reward produces the highest success rates, with GPT-4o achieving 71.4%, followed by ChatGLM-6B at 53.6%, and LLaMA2-7B at 32.1%.

4.5.4 Discussion

These results demonstrate the modified framework successfully increases the accuracy and model’s ability to generalize across tasks, particularly when incorporating both context and reward. These results highlight the impact of incorporating context and reward, with GPT-4o achieving the highest performance boost, reaching 71.4% success rates. It also showed that our modification to the framework have successfully improved the tasks accuracy on the provided dataset.

4.6 Error Analysis

We will analyze our best result from ChatGPT-4o. The performance of GPT-4o was assessed across multiple web navigation tasks. While the overall accuracy across all tasks is approximately 71.4%, a closer look at the results reveals areas of strength and weakness.

Strengths GPT-4o achieved 40 correct tasks out of the 56 tasks, demonstrating its capability in straightforward interactions like click-button, choose-date, book-flight, and navigate-tree. Tasks involving simple user interactions and structured webpage elements were executed effectively.

Weaknesses The model failed in 15 tasks. Input-based tasks, such as enter-password, enter-text, and enter-text-dynamic, indicate issues in handling precise text input operations. Advanced interaction tasks, such as count-shape, guess-number, and login-user, often require reasoning or dynamic decision-making. Context-sensitive tasks, such as use-autocomplete and use-spinner, may involve interactions reliant on asynchronous or dynamic web components.

Specific Observations Tasks like multi-orderings and social-media-some, which involve multi-step or partially complete operations, were challenging, resulting in a 0.00 score. The model performed inconsistently in email-related tasks, scoring perfectly in email-inbox-forward-nl but failing in variations like email-inbox-nl-turk, suggesting sensitivity to linguistic or structural variations in task setup. Performance was flawless in simpler interactive tasks (e.g., click-checkboxes and click-collapsible) but struggled with more dynamic elements (e.g., login-user-popup and use-spinner).

GPT-4o failed tasks that required handling dynamic or asynchronous elements (e.g., use-spinner) or complex reasoning and multi-step decision-making (e.g., login-user, multi-orderings). These failures indicate limitations in contextual understanding and adaptability to intricate or real-time interactive scenarios.

5 Future Directions

There are several directions for further exploration and enhancement of this topic. Firstly, curating or identifying resource-efficient fine-tuning datasets was a key challenge encountered - the scarcity of publicly available HTML fine-tuning datasets that align with our resource constraints. Future efforts could focus on curating or identifying resource-friendly datasets tailored for HTML-based tasks. Additionally, exploring smaller, pre-trained language models fine-tuned specifically on HTML tasks may provide a promising direction for developing lightweight yet effective agents.

It is also possible to explore more aspects of hyperparameter optimization. While our current work includes some improvements in hyperparameter tuning, further exploration of parameters like top-k sampling and learning rates could help refine model performance. Using automated tuning techniques such as Bayesian optimization, could systematically identify optimal configurations.

Lastly, while this work focuses on the MiniWoB++ dataset (Humphreys et al., 2022), future research could explore extending the methodology to other web navigation datasets or tasks requiring interaction with different web interfaces. Additionally, developing generalized adaptation frameworks or modular reward structures that can be easily transferred across datasets would help test the robustness and versatility of the models.

6 Conclusion

Our study demonstrates the effectiveness of incorporating context and reward modification into LLM-based web navigation agents. Our modifications significantly improved task accuracy, with GPT-4o achieving the highest success rate, outperforming other models in both baseline and modified configurations. While GPT-4o excels in accuracy and generalization, it incurs a trade-off with longer runtime. LLaMA2-7B and ChatGLM-6B, on the other hand, offer faster runtimes but show limited performance gains from context and reward integration. These results highlight the potential of tailored frameworks and optimization strategies in advancing autonomous web navigation, setting a foundation for future research on lightweight and efficient solutions.

References

- Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, and et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*.
- Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. 2024. Mind2web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems*, 36.
- Team GLM, :, Aohan Zeng, Bin Xu, Bowen Wang, Chenhui Zhang, Da Yin, Dan Zhang, Diego Rojas, Guanyu Feng, Hanlin Zhao, Hanyu Lai, Hao Yu, Hongning Wang, Jiada Sun, Jiajie Zhang, Jiale Cheng, Jiayi Gui, Jie Tang, Jing Zhang, Jingyu Sun, Juanzi Li, Lei Zhao, Lindong Wu, Lucen Zhong, Mingdao Liu, Minlie Huang, Peng Zhang, Qinkai Zheng, Rui Lu, Shuaiqi Duan, Shudan Zhang, Shulin Cao, Shuxun Yang, Weng Lam Tam, Wenyi Zhao, Xiao Liu, Xiao Xia, Xiaohan Zhang, Xiaotao Gu, Xin Lv, Xinghan Liu, Xinyi Liu, Xinyue Yang, Xixuan Song, Xunkai Zhang, Yifan An, Yifan Xu, Yilin Niu, Yuntao Yang, Yueyan Li, Yushi Bai, Yuxiao Dong, Zehan Qi, Zhaoyu Wang, Zhen Yang, Zhengxiao Du, Zhenyu Hou, and Zihan Wang. 2024. *Chatglm: A family of large language models from glm-130b to glm-4 all tools*. *Preprint*, arXiv:2406.12793.
- Izzeddin Gur, Hiroki Furuta, Austin Huang, Mustafa Safdari, Yutaka Matsuo, Douglas Eck, and Aleksandra Faust. 2024. *A real-world webagent with planning, long context understanding, and program synthesis*. *Preprint*, arXiv:2307.12856.
- Peter C Humphreys, David Raposo, Tobias Pohlen, Gregory Thornton, Rachita Chhaparia, Alistair Muldal, Josh Abramson, Petko Georgiev, Adam Santoro, and Timothy Lillicrap. 2022. A data-driven approach for learning to control computers. In *International Conference on Machine Learning*, pages 9466–9482. PMLR.
- Sheng Jia, Jamie Kiros, and Jimmy Ba. 2019. Domq-net: Grounded rl on structured language. *arXiv preprint arXiv:1902.07257*.
- Hanyu Lai, Xiao Liu, Iat Long Iong, Shuntian Yao, Yuxuan Chen, Pengbo Shen, Hao Yu, Hanchen Zhang, Xiaohan Zhang, Yuxiao Dong, et al. 2024. Autowebglm: Bootstrap and reinforce a large language model-based web navigating agent. *arXiv preprint arXiv:2404.03648*.
- Xiao Liu, Hanyu Lai, Hao Yu, Yifan Xu, Aohan Zeng, Zhengxiao Du, Peng Zhang, Yuxiao Dong, and Jie Tang. 2023. *Webglm: Towards an efficient web-enhanced question answering system with human preferences*. *Preprint*, arXiv:2306.07906.
- Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. 2021. Webgpt: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*.
- OpenAI. 2024. *Gpt-4 system card*. *OpenAI Technical Report*.
- OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa

- Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Łukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Łukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kopic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rajeev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O’Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael, Pokorny, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. 2024. [Gpt-4 technical report](#). *Preprint*, arXiv:2303.08774.
- Keiron O’Shea and Ryan Nash. 2015. [An introduction to convolutional neural networks](#). *Preprint*, arXiv:1511.08458.
- Tianlin Shi, Andrej Karpathy, Linxi Fan, Jonathan Hernandez, and Percy Liang. 2017. [World of bits: An open-domain platform for web-based agents](#). In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 3135–3144. PMLR.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. [Llama: Open and efficient foundation language models](#). *arXiv preprint arXiv:2302.13971*.
- Thijs Vogels, Octavian-Eugen Ganea, and Carsten Eickhoff. 2018. [Web2text: Deep structured boilerplate removal](#). In *Advances in Information Retrieval: 40th European Conference on IR Research, ECIR 2018, Grenoble, France, March 26-29, 2018, Proceedings 40*, pages 167–179. Springer.
- Zilong Wang, Mingjie Zhan, Xuebo Liu, and Ding Liang. 2020. [Docstruct: A multimodal method to extract hierarchy structure in document for general form understanding](#). *Preprint*, arXiv:2010.11685.
- Sean Wu, Michael Koo, Lesley Blum, Andy Black, Liyo Kao, Fabien Scalzo, and Ira Kurtz. 2023. [A comparative study of open-source large language models, gpt-4 and claude 2: Multiple-choice test taking in nephrology](#). *Preprint*, arXiv:2308.04709.
- Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. [Webarena: A realistic web environment for building autonomous agents](#). *Preprint*, arXiv:2307.13854.